

```
Backbone.history.start({pushState: true});
router.navigate('contacts/11', {trigger: true});
```

The routes hash lets you specify different locations within the app, and the method to be called when navigating to that URL. Here, we map the contacts path within the app to the `contactList` method, the second path has an `:id` parameter that is passed to its corresponding method, `contactDetail`. These methods have been blank here, but you could imagine what they'd do; initialising your models and collections, fetching remote data, and updating the UI.

After creating the router, and instantiating it in the `router` variable we call the code in Backbone that actually handles all this with the `Backbone.history.start` call. The `pushState` parameter set to `true` allows the browser to use neat urls such as `'http://myappsite.com/contacts'` instead of hash fragments such as `'http://myappsite.com/#contacts'`.

The final method call tells the app to go to a specific page within the app, and the trigger parameter set to `true` simply tells the Router to actually call the corresponding method instead of just changing the URL.

Putting the Pieces Together

You may have noticed that till now that we have used each component almost independently of other components. This is one of the strengths of Backbone. You can use bits and pieces of it in your own app without converting your whole app to it. Just want to use the Model and Collection modules to pull in data, go ahead! Like how Backbone Views work, use them without bothering with the rest. Or use the Router module alone to navigating within your app without bothering with Views, Models or Collections. This isn't to say these components don't work well together.

Frameworks like Marionette build on Backbone by extending existing Backbone modules and introducing new ones. For some they can be easier to start with than Backbone.



When creating an instance of a Backbone View, you can pass an instance of a model to bind the model to the view. For example: `var contactView = new ContactView({model: modelInstance});`. This model is then accessible inside the view. The view can then listen to changes in the model and update the display on page automatically!

All you will need to listen to the model change events in the `initialize` method of the View. You can also pull in data from the attached model to display in the view using the `toJSON` method we have mentioned above. Similarly, you can have Views bound to collections by passing the Collection instance as a parameter just like we have done with model above: `var addressBookView = new AddressBookView({collection: myAddressBook});`

Underscore, a dependency of Backbone includes a template engine that simplifies binding model data to HTML code, but Backbone does not require its use. In fact, it will work with any template engine you want to use, such as Jade or Handlebars.

Conclusion

There is no shortage of JavaScript libraries and frameworks out there that tell you what to do, how to lay out your code, but few like

Backbone that just lay back and let you handle things yourself. This might not make it better, or more suitable for all kinds of projects. In fact, you might find yourself implementing functionality that comes by default with many alternative libraries. However, Backbone is more amenable to use cases that lie slightly outside of the scope or comfort zone of more rigid frameworks.

Backbone can also be extended through plug-ins to support features like, saving to a local storage, integrating with different kinds of services, specialised views and models among other things.

If you're still hankering for more direction, there are frameworks such as Thorax and Marionette build on Backbone and give it a more opinionated direction.>

*lumbar support



*NewsBlur RSS Reader

>>NewsBlur is a great online RSS Reader that offers some unique features. It also happens to use Backbone, and is open-source.

www.newsblur.com



*Handlebars Template Engine

>>The handlebars template engine is a great option for handling templates in Backbone apps.

<http://handlebarsjs.com/>



*Codepen

>>Codepen is a good alternative to JS Fiddle and JS Bin if you are looking for one. It's got a neat inline error reporting feature.

<http://codepen.io/>